Name:Wu Sitong Student ID:1155253147

Agentic AI for Business and FinTech (FTEC5660) HW2 Part1

1. Task Objective

The goal of this assignment is to build an agentic AI system that verifies CV information by cross-referencing it against SocialGraph MCP profiles (simulated LinkedIn and Facebook data provided by the course). The system connects to the MCP server, uses search and profile tools to retrieve candidate profiles, applies an identity gate to avoid same-name mismatches, and assigns each CV a verification score in [0, 1]. Scores > 0.5 are classified as valid; scores ≤ 0.5 are classified as problematic.

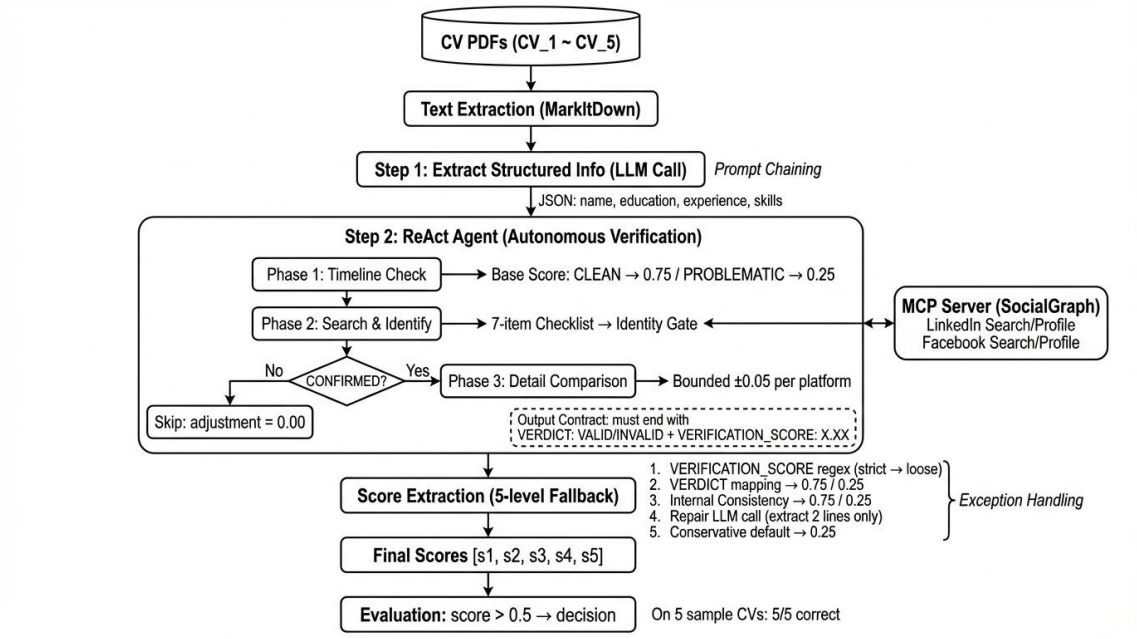
2. Approach & Architecture

I built the system as a two-step pipeline using **Prompt Chaining** combined with a **ReAct Agent**. Instead of asking one LLM call to do everything (which in my testing often led to unstable outputs or missed details), I split the task into a focused extraction step and an autonomous verification step.

Workflow

- **Step 1 — Structured Extraction (Prompt Chaining)**: A single LLM call reads the raw CV text and extracts key information (name, education, work experience, skills, locations) into a fixed JSON schema. This ensures the agent in the next step works with clean, structured data rather than parsing raw PDF text on its own.
- **Step 2 — ReAct Agent (Autonomous Verification)**: A ReAct agent receives both the structured JSON and the original CV text, then autonomously decides which MCP tools to call. It searches LinkedIn and Facebook for the candidate, confirms identity using a multi-signal checklist, and compares CV claims against social media profiles. The agent works in three phases: Timeline Check, Search & Identify, and Detail Comparison.
- **Score Extraction (Multi-level Fallback)**: After the agent completes, a multi-level parsing chain extracts the final score from the agent's output. This handles cases where the agent doesn't produce perfectly formatted output.

Figure 1: System Architecture — CV Verification Pipeline



3. Methodological Details

3.1 Step 1: CV Information Extraction

Each CV is converted from PDF to text using MarkItDown, then sent to Gemini 2.5 Pro with a prompt that forces JSON output in a fixed schema: {name, locations, education, experience, skills}. If the model returns code fences or extra text, the code strips them and parses the JSON. If parsing fails entirely, a safe default is returned to prevent pipeline crashes.

3.2 Step 2: Three-Phase Verification

The ReAct agent follows a structured system prompt with three phases:
Phase 1 — Timeline Check (base score): The agent checks the CV's own dates for internal contradictions. Two full-time jobs overlapping by more than one year, or an end date in 2027+, marks the CV as PROBLEMATIC (base = 0.25). Everything else is CLEAN (base = 0.75). I chose 0.75/0.25 specifically because the evaluation threshold is 0.5 — this creates a ±0.25 margin from the decision boundary, making the system robust to tool noise, LLM variability, and minor social media mismatches.
Phase 2 — Search & Identify (identity gate): The agent searches LinkedIn and Facebook using the candidate's name, picks the closest matching profile, and runs a multi-signal identity checklist (covering school, company, job title, city, industry, years, and skills). If at least 2 signals match, the profile is CONFIRMED and proceeds to Phase 3. Otherwise it is UNCONFIRMED — no social media adjustment is applied, and the base score is kept as-is. This "identity gate" prevents false matches from corrupting the score. According to the TA's Q&A clarification, every CV corresponds to a real profile in the MCP database (possibly with injected inconsistencies), so the agent always tries to pick the closest match rather than giving up if there is no exact match.
Phase 3 — Detail Comparison (bounded adjustment): For CONFIRMED profiles, the agent compares all details. Supporting evidence (consistent titles, matching dates, aligned education) adds +0.05 per platform. Contradicting evidence (completely different school, wildly different seniority) subtracts -0.05 per platform. Minor differences like title wording or LinkedIn not being updated are ignored. Hard bounds are enforced: CLEAN scores stay within [0.60, 0.90], PROBLEMATIC scores stay within [0.10, 0.40].

3.3 Output Contract

The system prompt requires the agent to end with exactly two lines: VERDICT: VALID/INVALID and VERIFICATION_SCORE: X.XX (two decimal places). This contract makes downstream parsing reliable.

3.4 Exception Handling (Multi-level Fallback)

Despite the output contract, LLM outputs can be unpredictable. I implemented a multi-level

fallback chain for score extraction:

1. Regex extraction for VERIFICATION_SCORE (strict → loose)
2. VERDICT keyword mapping → 0.75 (VALID) / 0.25 (INVALID)
3. Internal consistency mapping → 0.75 (CLEAN) / 0.25 (PROBLEMATIC)
4. Repair LLM call — a lightweight prompt that extracts only the two required output lines from the agent's transcript
5. Conservative default → 0.25 (treat as suspicious if all parsing fails)

This ensures the system never crashes and always produces a score, even if the agent's response format is messy. The conservative default of 0.25 means that in the worst case, a CV is flagged as problematic rather than accidentally validated.

3.5 Tool Usage Strategy

The agent has access to 6 MCP tools via the SocialGraph server. The system prompt includes explicit tool rules to prevent common errors:

- search_facebook_users only accepts q, limit, fuzzy — never pass location parameters.
- search_linkedin_people accepts q, location, industry, limit, fuzzy.

The agent always starts with a broad name-only search, then narrows down based on overlapping information.For each CV, the agent typically makes 4 tool calls: search LinkedIn, get LinkedIn profile, search Facebook, get Facebook profile.

4. Sample Verification Results

The system was evaluated on 5 provided CVs' Results:

CV	Candidate	Score	Verdict	Timeline	LinkedIn	Facebook
CV_1	John Smith	0.75	VALID	CLEAN (0.75)	CONFIRMED, supportive (+0.05)	CONFIRMED, contradiction (-0.05)
CV_2	Minh Pham	0.80	VALID	CLEAN (0.75)	CONFIRMED, supportive (+0.05)	UNCONFIRMED (0.00)
CV_3	Wei Zhang	0.85	VALID	CLEAN (0.75)	CONFIRMED, supportive (+0.05)	CONFIRMED, supportive (+0.05)
CV_4	Rahul Sharma	0.20	INVALID	PROBLEMATIC (0.25)	CONFIRMED, contradiction (-0.05)	UNCONFIRMED (0.00)
CV_5	Rahul Sharma	0.15	INVALID	PROBLEMATIC (0.25)	CONFIRMED, contradiction (-0.05)	CONFIRMED, contradiction (-0.05)

Detailed findings are from the agent's generated verification report for each CV. Note that CV_4 and CV_5 share the same candidate name in the provided dataset.

Evaluation: Using the assignment's decision rule (score > 0.5), all 5 decisions matched ground truth: decisions = [1, 1, 1, 0, 0], correct = 5/5, **final_score = 1.0**.

5. Alignment with Assignment Requirements

Requirement	How my system handles it
Connect to MCP server	Uses MultiServerMCPClient with HTTP transport to SocialGraph
Extract information from CVs	MarkItDown for PDF→text; Prompt Chaining Step 1 for structured JSON extraction
Search and match LinkedIn/Facebook profiles	ReAct agent autonomously calls search + profile tools
Detect discrepancies	Phase 1 catches timeline issues; Phase 3 catches social media contradictions
Generate verification report	Agent produces structured report with checklist, scoring breakdown, and verdict
Output format	Returns scores list of 5 floats in [0, 1]

6. Limitations & Reflection

LLM non-determinism.
Even with temperature=0, Gemini 2.5 Pro can produce slightly different outputs across runs. The same CV might score 0.75 in one run and 0.80 in another, depending on how the agent interprets minor discrepancies. The multi-level fallback and hard bounds help keep scores in a stable range, but exact reproducibility is not guaranteed.

Facebook data quality.
In some cases, the Facebook profile for a candidate contains different information from LinkedIn (e.g., different employer or education level). This is by design (the TA mentioned injected inconsistencies), meaning the agent sometimes applies a -0.05 penalty for a generally CLEAN CV. I deliberately set the detail comparison penalty to a strict +/- 0.05 to ensure that these minor injected inconsistencies wouldn't override the primary timeline verification. This trade-off prioritizes overall pipeline stability over hyper-sensitivity

Identity confirmation threshold.
Setting the confirmation threshold at 2 matches works well for this dataset, but could be too loose or too strict for other datasets. A more sophisticated approach might weight different signals differently (e.g., school match is stronger evidence than city match).

Single-candidate selection.
The agent picks only the best-matching candidate from search results. If the MCP database contained many people with similar names and backgrounds, the agent might pick the wrong one. For this assignment's dataset, this hasn't been an issue.

7. Key Code Snippets

Simplified snippets are shown below; the actual implementation includes async calls, additional sanitization, and retry logic with exponential backoff.

Prompt Chaining Step 1 — CV extraction function:

```
def extract_cv_info(cv_text: str) -> dict:
    prompt = f"""You are a CV parser. Extract the following
    from the CV below and return ONLY valid JSON..."""
    response = llm.invoke(prompt)
    content = response.content.strip()
    # Strip code fences if present
```

```
    if "```json" in content:
        content = content.split("```json")[1].split("```")[0]
    return json.loads(content.strip())
```

Multi-level fallback — score extraction:

```
async def robust_extract_score(llm_instance, messages):
    score, verdict, consistency, report = extract_from_messages(messages)
    if score is not None:        return score, report, "VERIFICATION_SCORE"
    if verdict == "VALID":       return 0.75, report, "VERDICT=VALID"
    if verdict == "INVALID":     return 0.25, report, "VERDICT=INVALID"
    if consistency == "CLEAN":   return 0.75, report, "Consistency=CLEAN"
    if consistency == "PROBLEMATIC": return 0.25, report, "Consistency=PROBLEMATIC"
    # Repair call: ask LLM to extract just the two output lines
    r_score, r_verdict, r_text = await repair_call(llm_instance, messages)
    if r_score is not None:      return r_score, r_text, "repair_call"
    return 0.25, "FALLBACK", "fallback_0.25"  # conservative default
```

8. Conclusion

In conclusion, splitting the CV verification task into a Prompt Chaining extraction step and a ReAct verification step proved to be a highly effective strategy. It minimized hallucination by giving the agent clean data to work with, allowing it to focus purely on logical tool calling and cross-referencing via the SocialGraph MCP. The combination of an identity gate, bounded scoring, and a multi-level fallback chain resulted in a robust pipeline that successfully achieved 5/5 correct decisions (final_score = 1.0) on the provided dataset, cleanly separating valid CVs (scoring 0.75–0.85) from problematic ones (scoring 0.15–0.20).